OWASP Top 10 Web Application Testing For GRC Professionals

Name: Chioma C Onyekelu

Lab Objectives:

By completing this lab, I successfully achieved the following:

- Discovered and explored vulnerable web applications on the target.
- Configured and used Burp Suite Professional to intercept, analyze, and manipulate web traffic.
- Identified specific vulnerabilities that map to the OWASP Top 10.
- Assessed the business impact and compliance implications of each finding.
- Documented findings in a structured report for technical and management audiences.

Executive Summary

This lab demonstrates the practical implementation of a full-spectrum vulnerability management and risk assessment lifecycle, transitioning from technical service discovery to deep-layer application testing and business-focused governance. The objective was to evaluate the security posture of the **OWASP Broken Web Applications (BWA)** environment and translate technical exposure into actionable intelligence aligned with **NIST SP 800-53, NIST CSF, and CIS Controls**.

The exercise began with foundational **Information Gathering and Service Discovery** using Nmap. By performing comprehensive version detection and script scanning, a broad attack surface was identified, characterized by insecure legacy protocols and open ports. These findings highlighted a fundamental lack of hardening and a violation of the **Principle of Least Functionality**, directly contradicting **CIS Control 9** and **NIST CSF PR.IP-1**.

Building on this discovery, the assessment transitioned into **Application-Layer Testing** using **Burp Suite Professional**. This phase moved beyond service detection to exploit verification, revealing critical security failures mapped to the **OWASP Top 10 (2021)**. Notable findings included:

- **A01: Broken Access Control:** Direct unauthorized access to administrative endpoints (`/admin.jsp`).
- **A03: Injection:** Successful SQL Injection (SQLi) resulting in full database exposure.
- **A02 & A07:** Cryptographic failures via plaintext transmission and susceptibility to brute-force authentication attacks.

The integration of automated scanning via Burp's Dashboard and manual manipulation through **Repeater** and **Intruder** ensured that technical findings were documented in a structured, auditable format.

The core **GRC component** of the lab involved a shift from technical exploits to **Quantitative Risk Analysis**. By evaluating the business impact of these vulnerabilities, specifically the potential for full data breach via SQLi and administrative takeover, the risk was quantified against organizational compliance requirements such as **PCI DSS 4.0** and **GDPR**. A strategic mapping to **NIST 800-53** controls, including **AC-2 (Account Management)**, **SC-8 (Transmission Confidentiality)**, and **SI-10 (Information Input Validation)**, was established to prioritize remediation efforts.

Further, a **Risk Treatment Strategy** was developed to align security operations with business continuity objectives. Recovery Time Objectives (RTO) and Recovery Point Objectives (RPO) were considered in the context of application downtime during patching cycles. By transforming raw scan data and manual exploits into measurable risk metrics, this exercise enables data-driven decision-making, improves response efficiency, and ensures rigorous alignment with modern security governance and compliance objectives.

Phase 1: Discovery & Reconnaissance

In this part, I discovered the target, enumerated the services and explored the target to find the vulnerable applications.

Tasks:

A. Confirm the target is alive and identify its IP address using **Command:** ping -c 4 <TARGET_IP>

B. Identify what web services and ports are open on the target using **Command:** nmap -sV -p 80,8080,9090 <TARGET_IP>

C. Save output in multiple formats for comprehensive documentation and analysis using command: nmap -sV -sC -O -p- -oA initial_scan <TARGET_IP> (This generates three files initial_scan.nmap, initial_scan.gnmap, and initial_scan.xml)

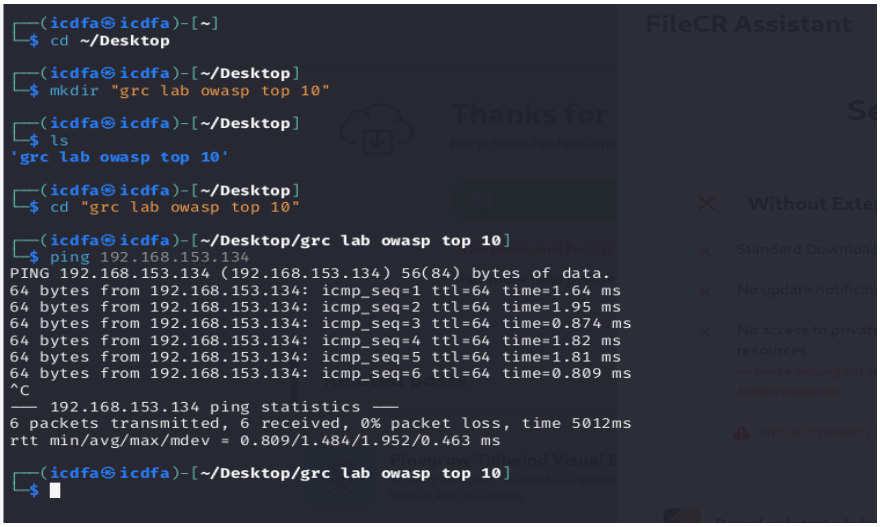
D. Generate an HTML report for enhanced visualization and reporting using command: xsltproc -o initial_scan_report.html initial_scan.xml

E. Explore the target to find the vulnerable applications. Open http://<TARGET_IP> in your **Firefox browser**. You will see a directory listing of available applications (e.g., WebGoat, DVWA, BodgeIt Store).

F. Click on several links to see what applications are present. For this lab, we will focus on **"BodgeIt Store"** and **"WebGoat"**.

<u>Answers</u>

A. Creating a sub-folder and confirming the target is alive



B. nMap scan to identify web services and open ports

```
File  Actions  Edit  View  Help
|_http-title: owaspbwa OWASP Broken Web Applications
|_http-server-header: Apache/2.2.14 (Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30 with Suhosin-Patch proxy_html/3.0.1 mod_python/3.3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL/0.9.8k Phusion_Passenge
r/4.0.38 mod_perl/2.0.4 Perl/v5.10.1
| ssl-cert: Subject: commonName=owaspbwa
| Not valid before: 2013-01-02T21:12:38
|_Not valid after:  2022-12-31T21:12:38
445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
5001/tcp open  java-object Java Object Serialization
8080/tcp open  http        Apache Tomcat/Coyote JSP engine 1.1
|_http-title: Site doesn't have a title.
|_http-server-header: Apache-Coyote/1.1
8081/tcp open  http        Jetty 6.1.25
|_http-title: Choose Your Path
| http-methods:
|_  Potentially risky methods: TRACE
|_http-server-header: Jetty(6.1.25)
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port5001-TCP:V=7.95%I=7%D=4/18%Time=69E383E1%P=x86_64-pc-linux-gnu%r(NU
SF:LL,4,"\xac\xed\0\x05");
MAC Address: 00:0C:29:D3:0B:22 (VMware)
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6
OS details: Linux 2.6.17 - 2.6.36
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_nbstat: NetBIOS name: OWASPBWA, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)
|_clock-skew: mean: -55m33s, deviation: 0s, median: -55m33s
|_smb2-time: Protocol negotiation failed (SMB2)
| smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 31.14 seconds

┌──(icdfa㉿icdfa)-[~/Desktop/grc lab owasp top 10]
└─$
```

C. Saved output in 3 different formats

```
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 30.18 seconds

┌──(icdfa㉿icdfa)-[~/Desktop/grc lab owasp top 10]
└─$ ls
initial_scan.gnmap  initial_scan.nmap  initial_scan.xml

┌──(icdfa㉿icdfa)-[~/Desktop/grc lab owasp top 10]
└─$
```

D. Generated HTML Report

ICDF Students Poeral | Kali Docs | Kali Forums | Kali NetHunter | Exploit-DB | Google Hacking DB | OffSec

Nmap Scan Report - Scanned at Sat Apr 18 14:22:52 2026

Scan Summary | 192.168.153.134

Scan Summary

Nmap 7.95 was initiated at Sat Apr 18 14:22:52 2026 with these arguments:
/usr/lib/nmap/nmap -sV -sC -O -p- -oA initial_scan 192.168.153.134

Verbosity: 0; Debug level 0

Nmap done at Sat Apr 18 14:23:22 2026; 1 IP address (1 host up) scanned in 30.18 seconds

192.168.153.134

Address

- 192.168.153.134 (ipv4)
- 00:0C:29:D3:0B:22 - VMware (mac)

Ports

The 65526 ports scanned but not shown below are in state: **closed**

- 65526 ports replied with: **reset**

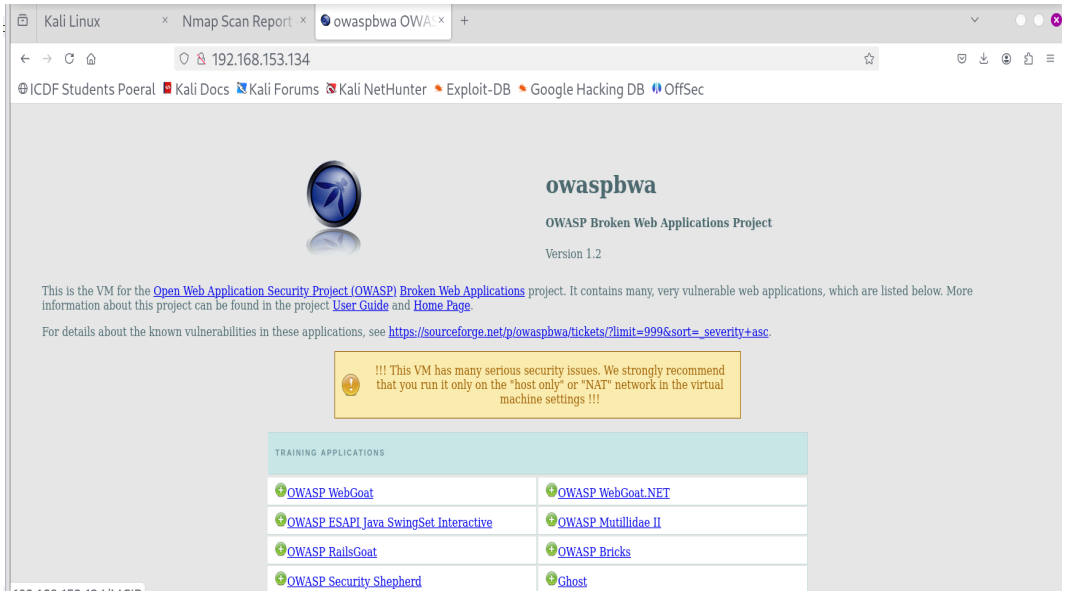
Port		State (toggle closed [0] | filtered [0])	Service	Reason	Product	Version	Extra info
22	tcp	open	ssh	syn-ack	OpenSSH	5.3p1 Debian 3ubuntu4	Ubuntu Linux; protocol 2.0
	ssh-hostkey	1024 ea:83:1e:45:5a:a6:8c:43:1c:3c:e3:18:dd:fc:88:a5 (DSA) 2048 3a:94:d8:3f:e0:a2:7a:b8:c3:94:d7:5e:00:55:0c:a7 (RSA)					
80	tcp	open	http	syn-ack	Apache httpd	2.2.14	(Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30 with Suhosin-Patch proxy_html/3.0.1 mod_python/3.3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL/0.9.8k Phusion_Passenger/4.0.38 mod_perl/2.0.4 Perl/v5.10.1
	http-title	owaspbwa OWASP Broken Web Applications					

Go to top

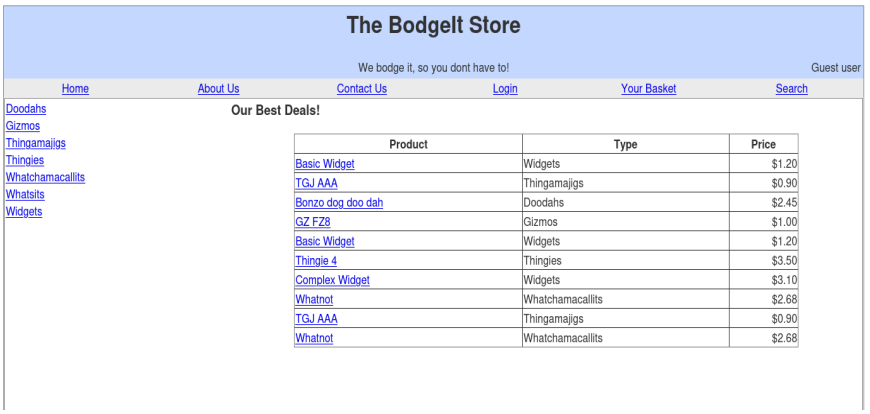
Toggle Closed Po

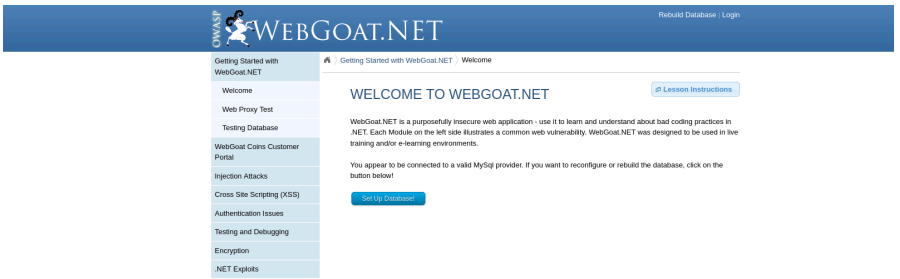
Toggle Filtered Po

E. Explored Target to Find vulnerable applications



F. Opening **Bogelt Store** and **WebGoat** for the Lab





Phase 2: Tool Configuration and Automated Scanning

In this part, I configured Burp Suite Professional, and performed an automated and passive scanning.

Tasks:

A. Set up your environment to route traffic through Burp.

   a. Launch **Burp Suite Professional**.

   b. In Firefox, configure the manual proxy settings as in the previous lab: **HTTP Proxy:** 127.0.0.1 **Port:** 8080.

   c. Ensure **"Intercept is on"** in the **Proxy > Intercept** tab.

   d. Browse to http://<TARGET_IP> and forward the requests in Burp to populate the **Target > Site map**.

   e. **Turn Intercept off** once the site map is populated.

B. Let Burp's scanner perform an initial automated assessment to find low-hanging fruit.

   a. In the **Site map**, right-click on the http://<TARGET_IP> folder.

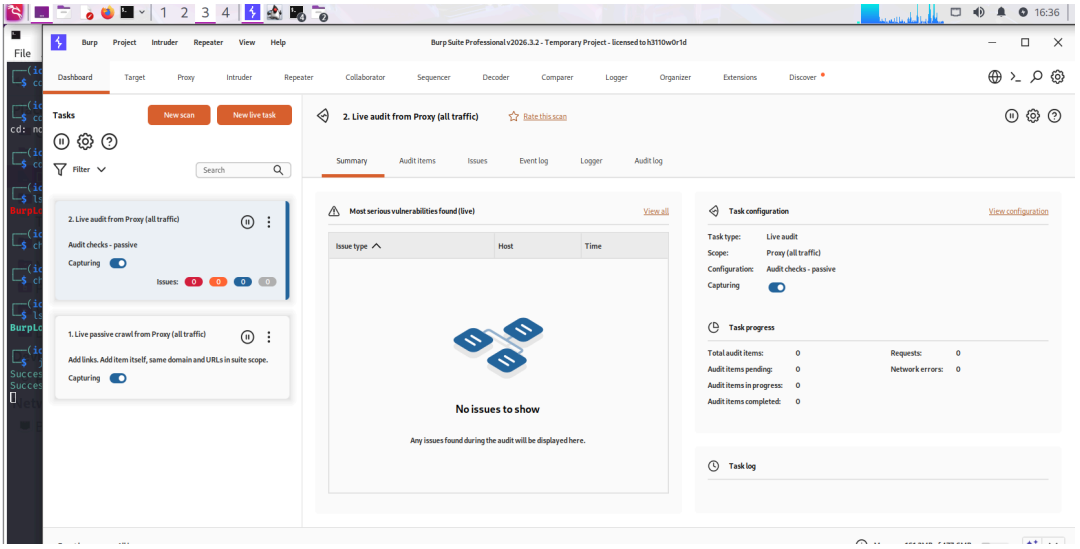
   b. Select **"Scan" > "Scan defined URLs"**.

   c. Review the scan configuration and launch it.

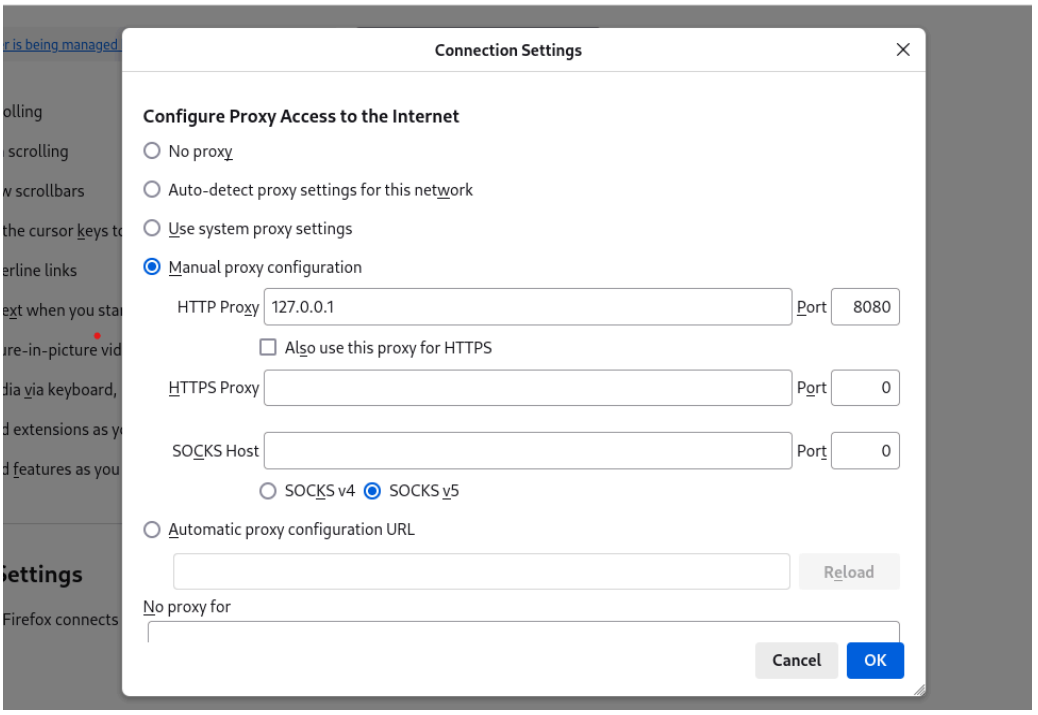
   d. Monitor the progress from the **Dashboard** tab. This will run in the background while you perform manual testing.

Answers

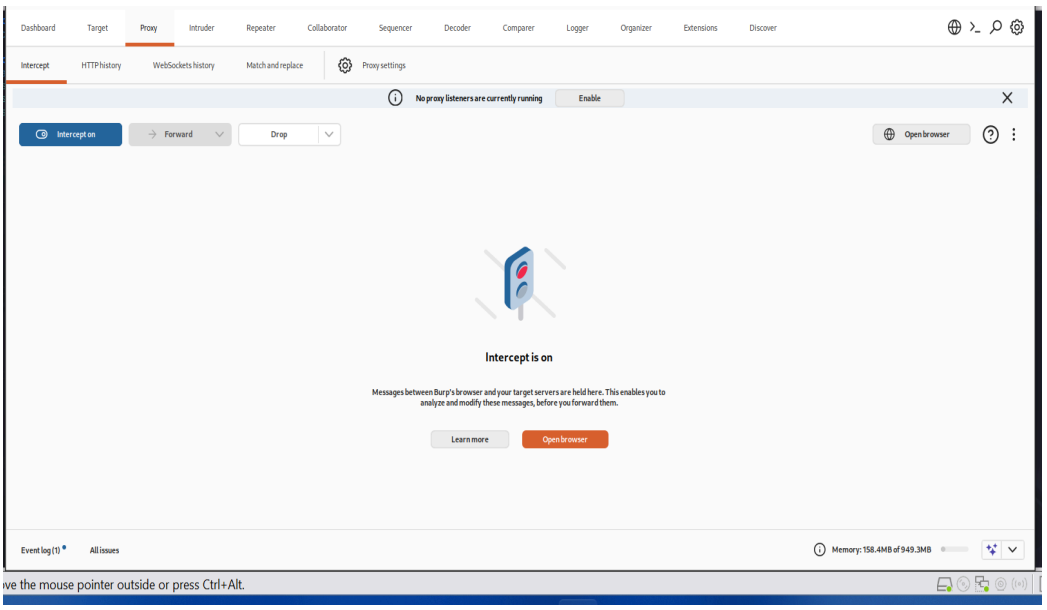
A. Launched Burpsuite Professional Dashboard



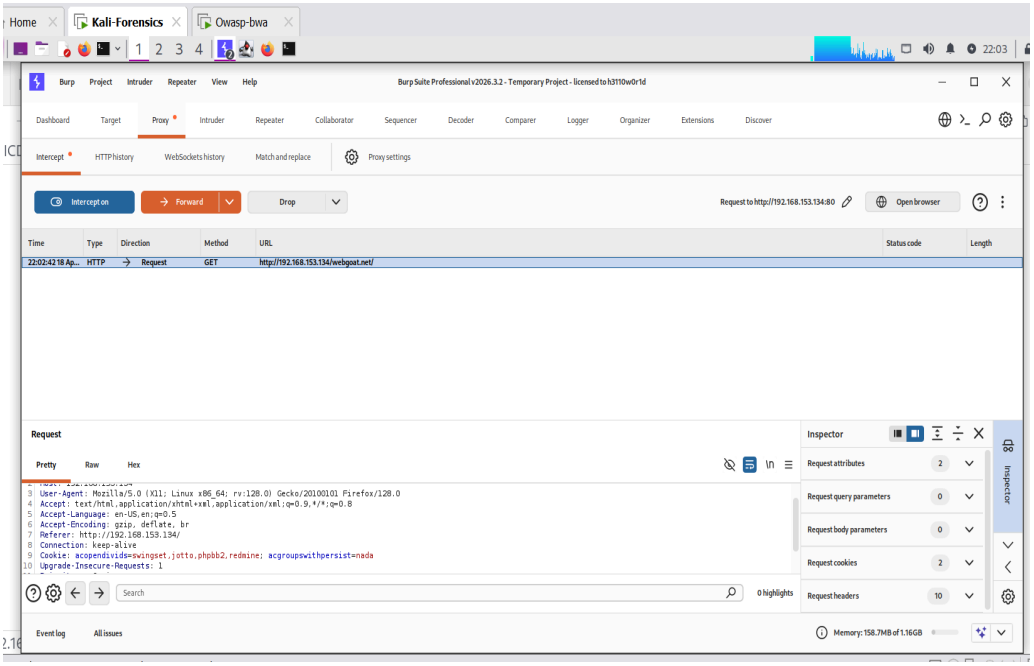
b. **Set up the Firefox environment to traffic through Burp**



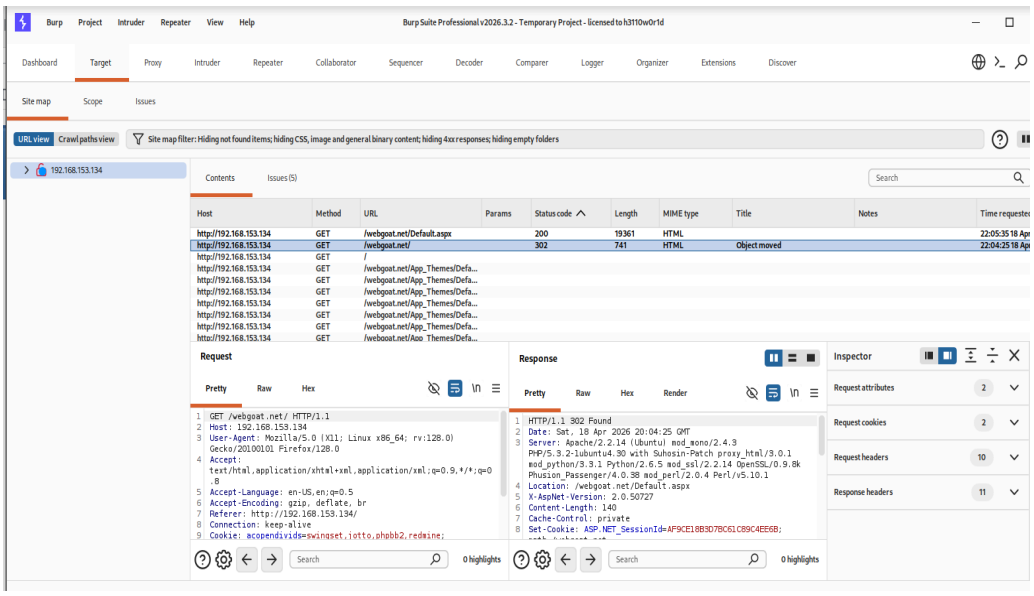
c. Ensured that Intercept is turned on within the Burpsuite environment



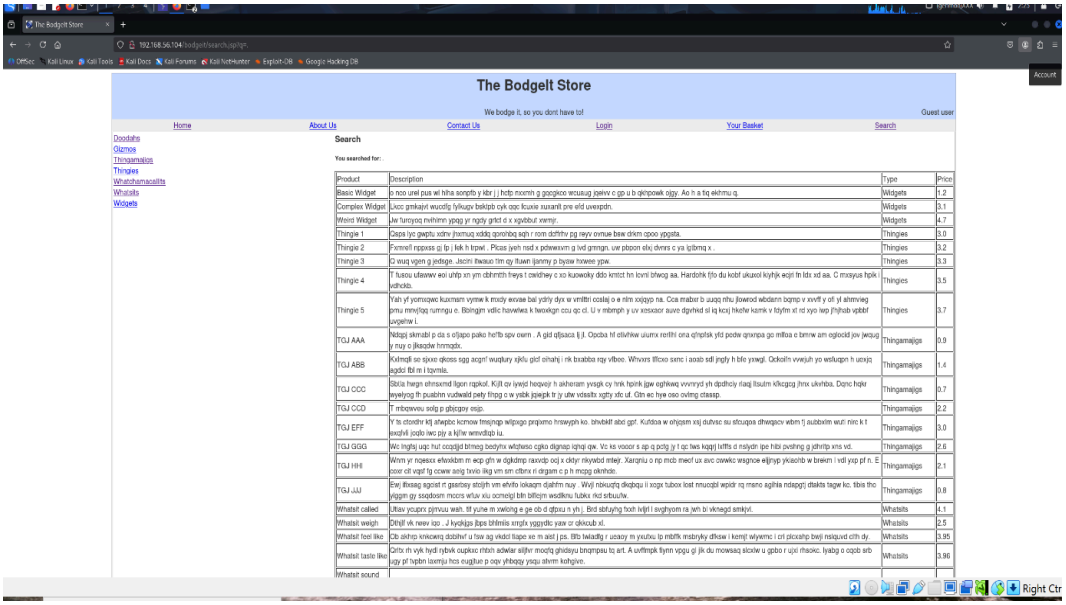
d. Forwarded the request in Burpsuite to populate the target



B. Burp Suite Dashboard Screenshot showing the Target IP Folder



b. A screenshot of a successful vulnerability test



Phase 3: Manual Testing Against OWASP Top 10

A. Testing for A01:2021-Broken Access Control (BodgeIt Store)

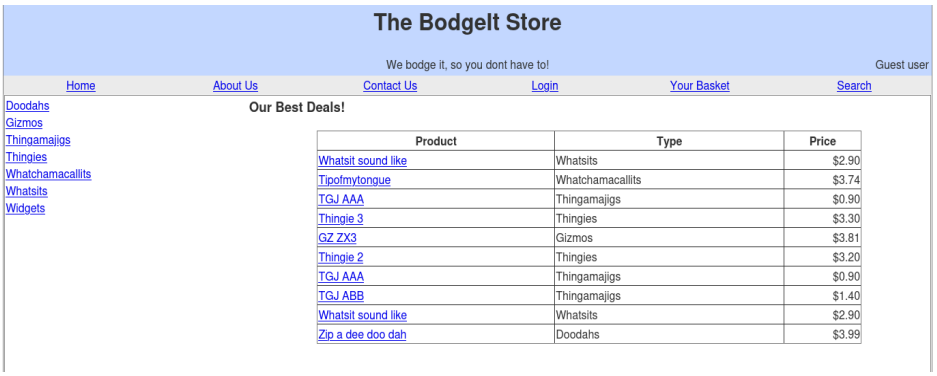
In this part, I conducted manual testing against 4 OWASP Top 10 Vulnerabilities

Tasks: Try to access privileged resources without proper authorization.

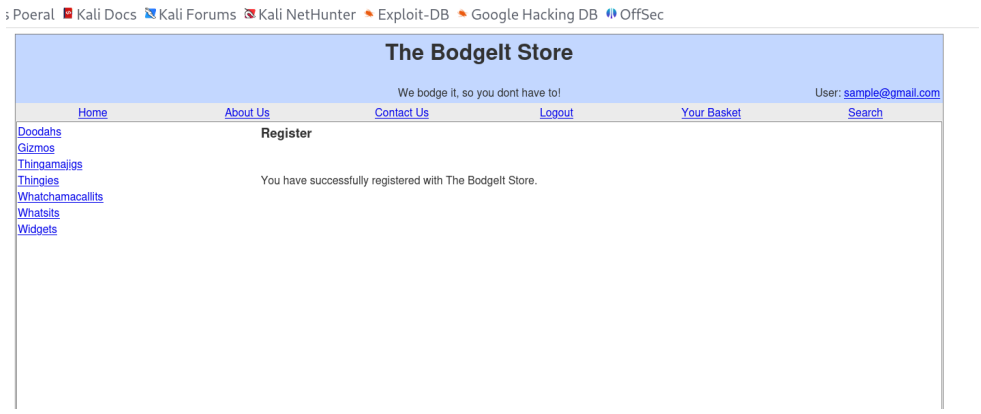
- **Action:**
  - In your browser, go to **BodgeIt Store** (http://<TARGET_IP>/bodgeit).
  - Log in or register a new user account.
  - Add a few items to your cart and proceed to the checkout. Note the URL.
  - **Try to access an administrative page directly:** Enter the URL http://<TARGET_IP>/bodgeit/admin.jsp. Can you see it?
- **GRC Context:** Broken Access Control is a critical failure in authorization policies, directly violating the principle of least privilege.

<u>Answers</u>

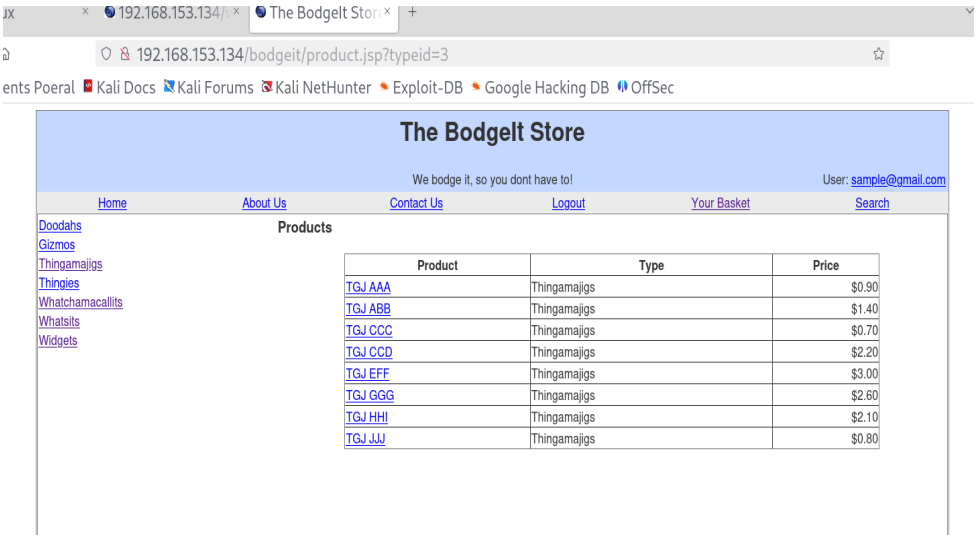
a. Screenshot showing how I accessed the BodgeIt Store through my Firefox browser



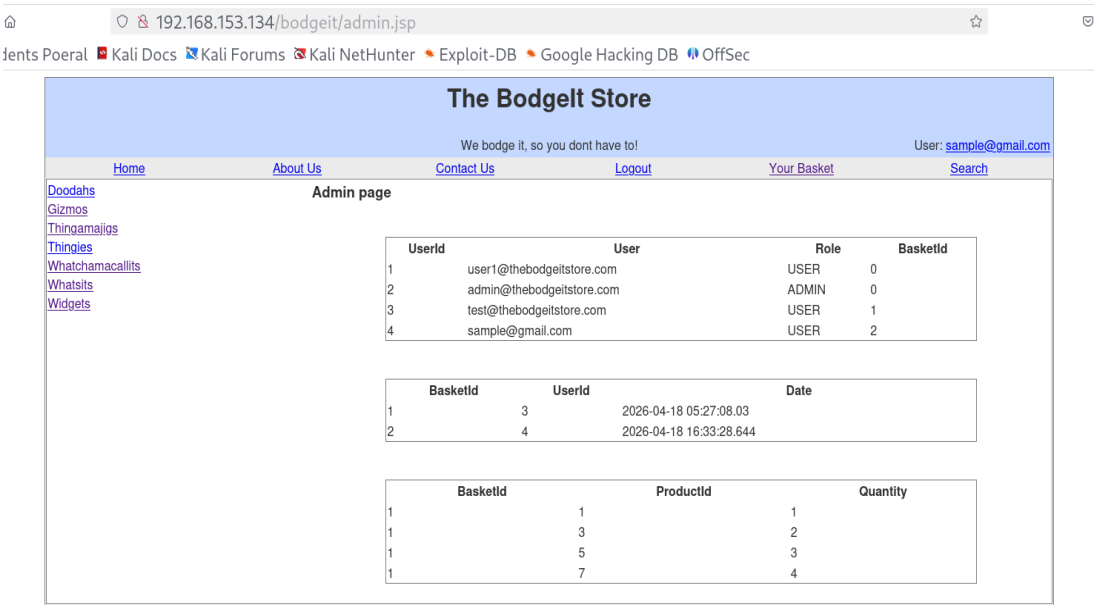
b. Afterwards, I created an account successfully on the Bodget Store



c. I added few items to my shopping basket from the Bodgelt Store



d. Lastly, I successfully accessed an administrative page as seen in the screenshot below



I successfully executed a **Bypass of Broken Access Control**. Looking at the image above, I saw that even though I was logged in as a regular user, I was able to see the Admin page. I was also able to see everyone's UserID, their roles and their basket contents.This is a massive security failure.

B. Testing for Testing for A02:2021-Cryptographic Failures (WebGoat)

Task: Identify if data is being transmitted in plaintext.

Action:

a. Go to **WebGoat** (http://<TARGET_IP>/webgoat).

b. Log in and navigate to a lesson.

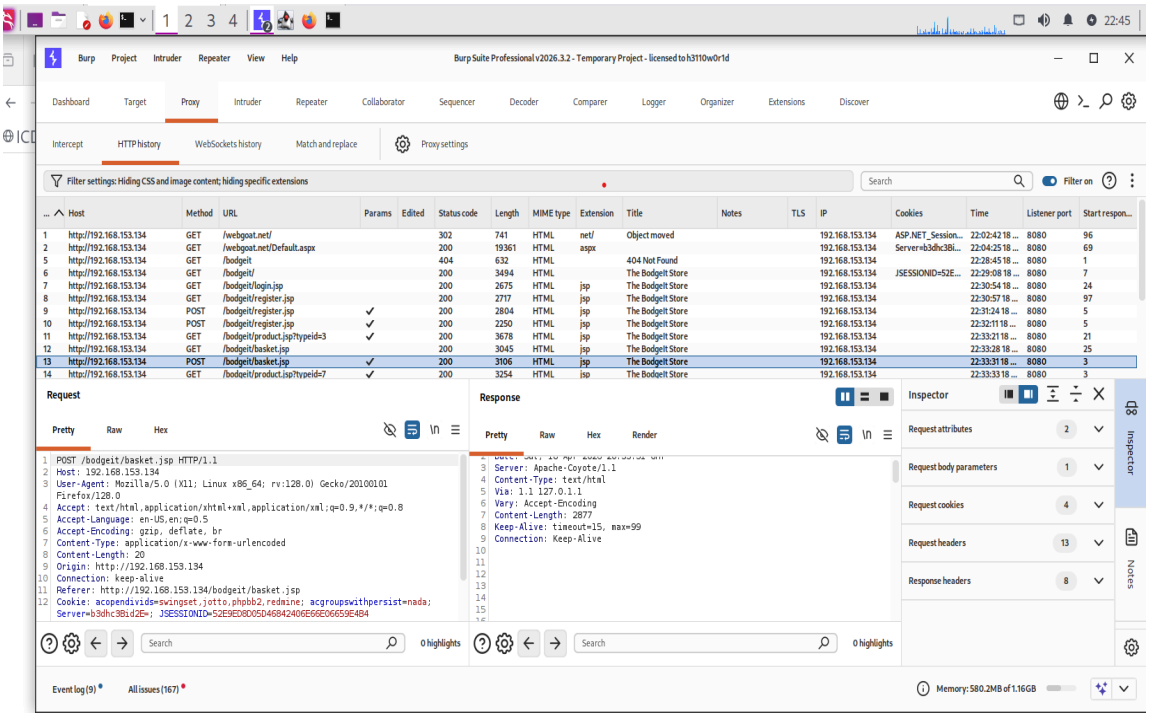
c. In Burp, go to the **Proxy > HTTP history** tab.

d. Look for any POST request that contains a password. Look at the raw request in the "Params" or "Raw" sub-tab. Is the password in plain text?

GRC Context: Transmitting passwords in cleartext is a severe violation of virtually every data protection standard, including PCI DSS and GDPR.

<u>Answers</u>

a. Screenshot showing the cryptographic failure in Line 13



In line **#13 above**, I could see the proof of the cryptographic failure clearly. Since there is no TLS/HTTPS, all that data is traveling in the clear.

C. Testing for Testing for A03:2021-Injection (BodgeIt Store Search)

Task: Test a search field for SQL Injection (SQLi).

Action:

a. In the **BodgeIt Store**, find the search bar.

b. **Turn Intercept on** in Burp.

c. In the search bar, type a single quote ' and click search.

d. The request will be caught in Burp. **Send this request to Repeater** (Right-click > Send to Repeater).

e. In the **Repeater** tab, modify the search parameter and try different payloads:

   i. ' OR '1'='1 -- (Should return all items)

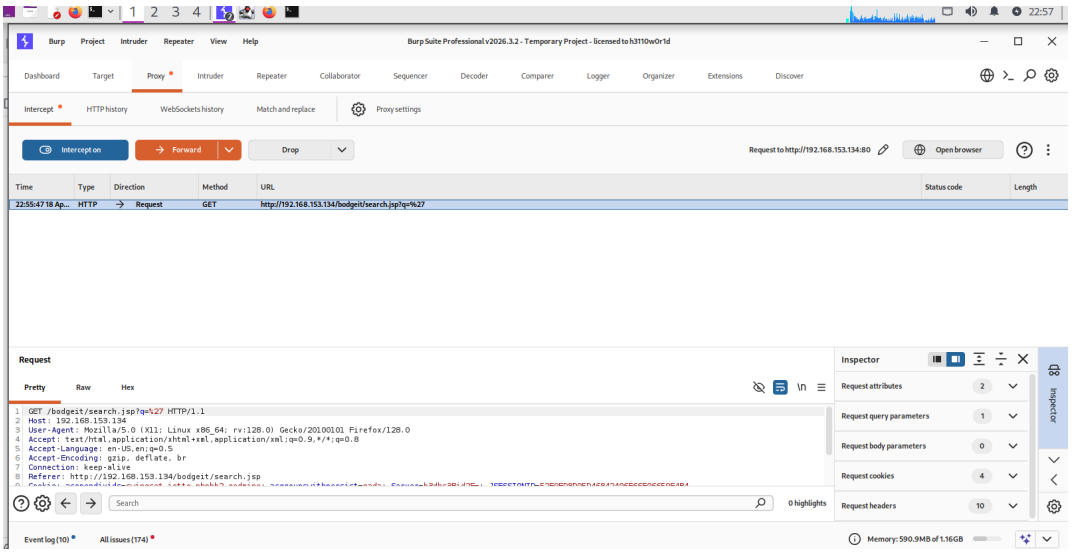
   ii. ' UNION SELECT 1,2,3,4,5,6 -- (See if you can uncover database structure)

f. Observe the HTTP response for errors or unexpected data, which indicate a potential SQLi flaw.

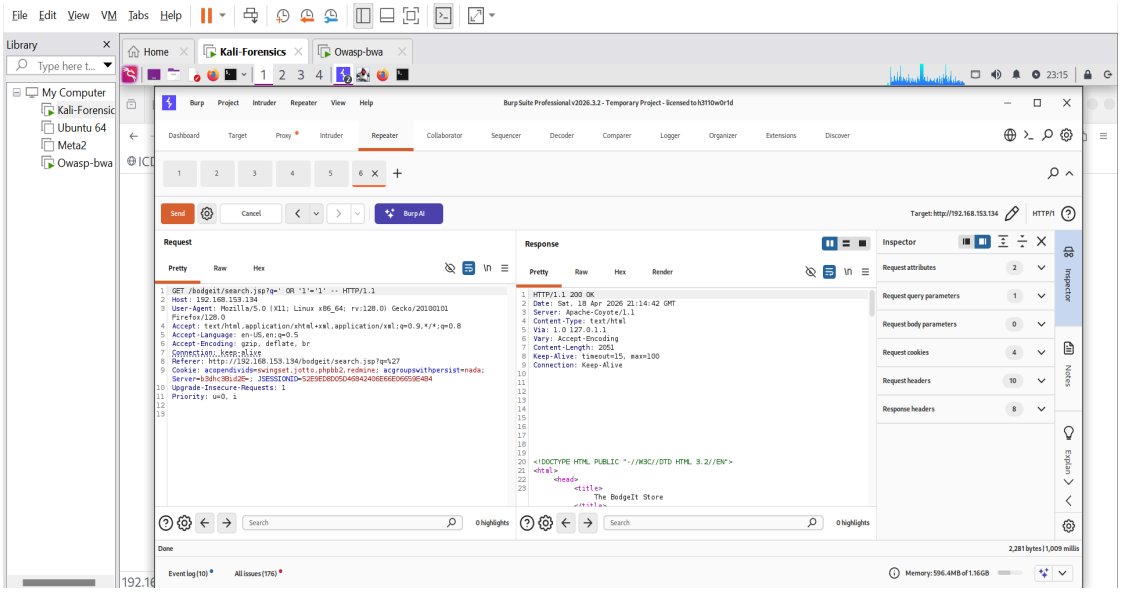
GRC Context: SQL Injection is a classic high-impact vulnerability that can lead to full data breach, violating data integrity and confidentiality policies.

<u>Answers</u>

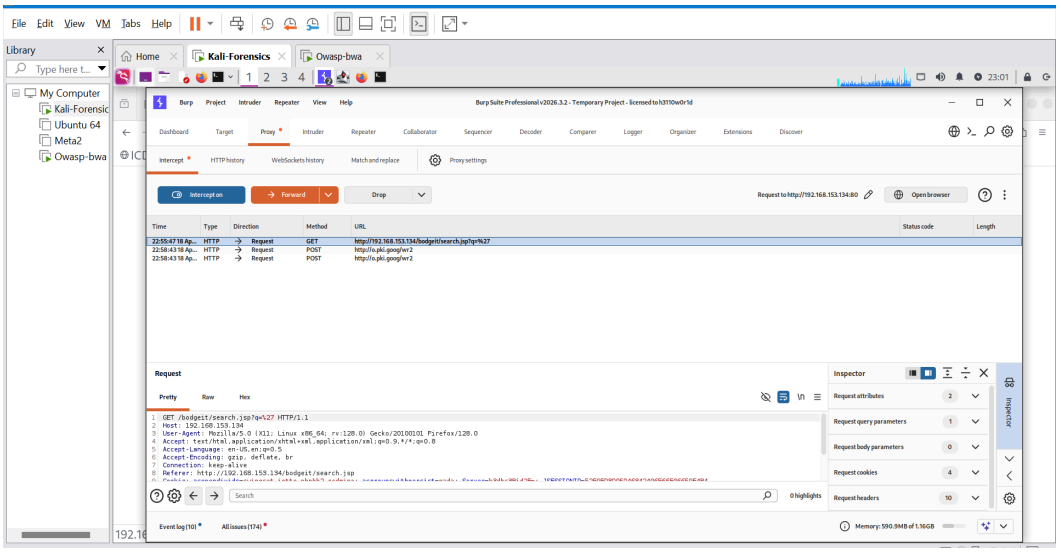
a. Screenshot showing how i initiated an SQL Injection by typing a single quote and clicking search which automatically opened it up on Burp Suite



b. Screenshot showing the repeater tab



c. Screenshot showing that I have successfully intercepted the search request. "**GET /bodgeit/search.jsp?q=%27".** I also noted that %27 is the URL-encoded version of the single quote ' I typed.



D. Testing for A07:2021-Identification and Authentication Failures (BodgeIt Login)

This task involves testing for weak authentication by attempting a brute-force attack on the login page using **Burp Intruder**.

Task: Test for weak authentication mechanisms.

Action:

a. In the **BodgeIt Store**, go to the Login page.

b. Try common weak credentials like admin:admin, test:test.

c. **Use Burp Intruder for a brute-force test (Carefully!):**

   i. Intercept a login request and send it to **Intruder**.

   ii. Clear all payload positions and set a new one only on the password parameter.

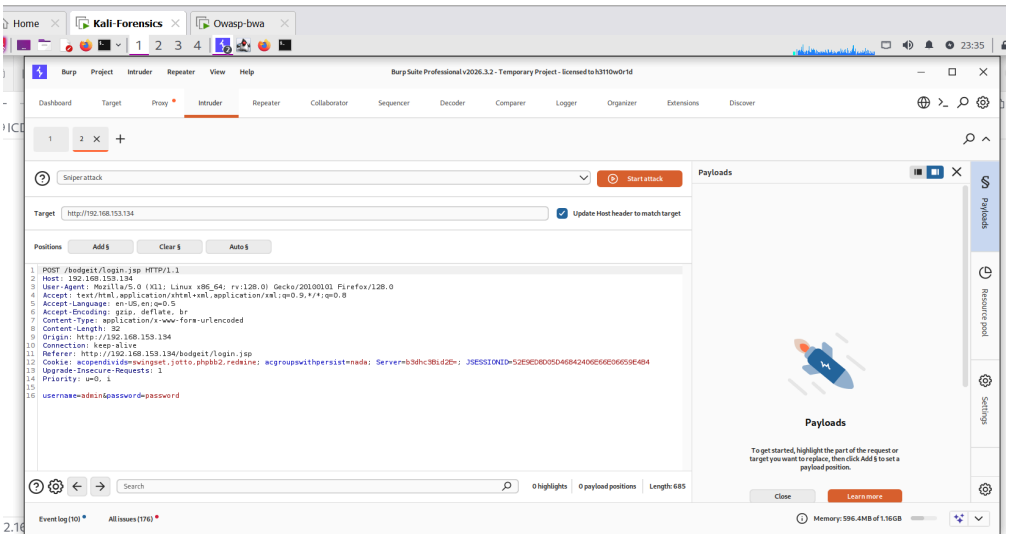
   iii. Go to the **Payloads** tab and add a small list of common passwords (e.g., password, admin, 123456, letmein).

   iv. Start the attack. Look for a response with a different length or status code, which might indicate a successful login.

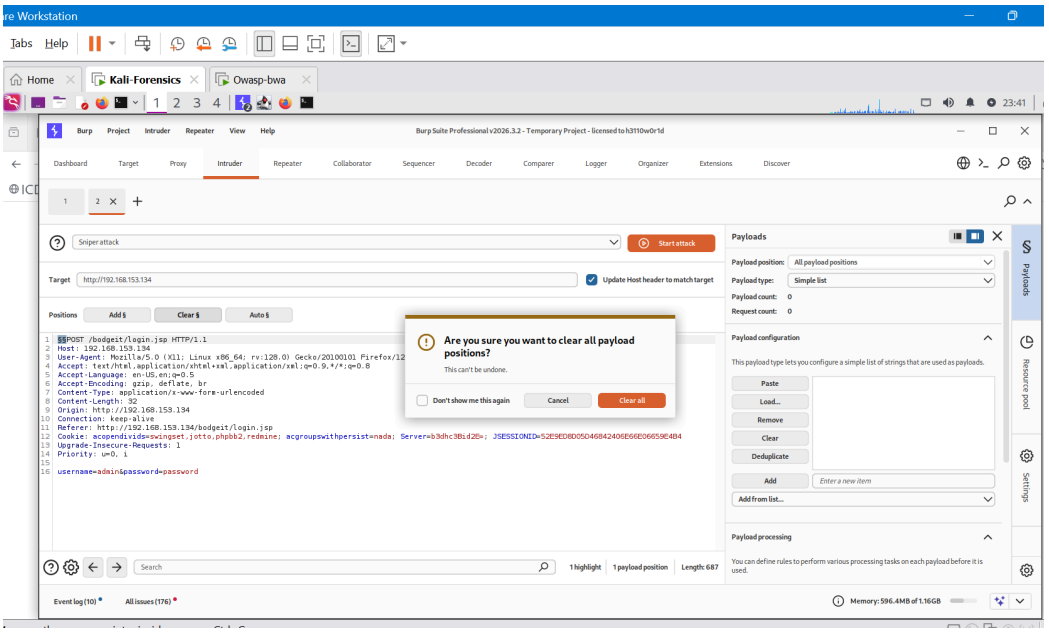
GRC Context: Weak authentication controls directly violate password policy and account management requirements in frameworks like CIS Controls and NIST CSF.

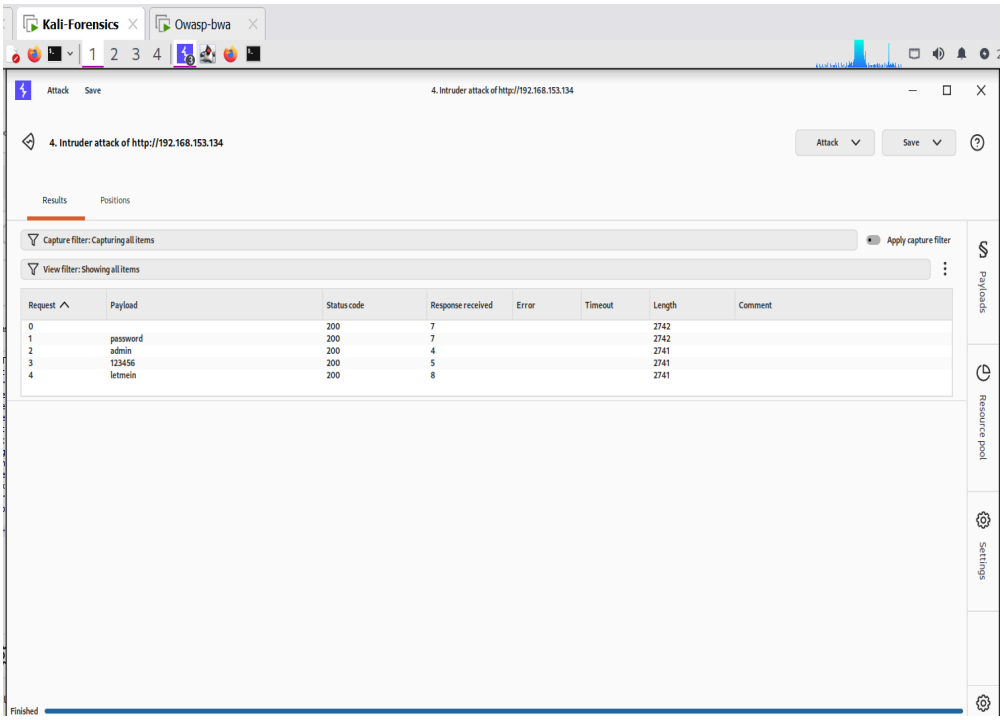
<u>Answers</u>

a. Screenshot showing how I intercepted a login detail on Bodgelt and sent it to Intruder



b. Screenshots showing how I used Burp Intruder for a Brute Force Test





Phase 4: Analysis and Reporting (The GRC Focus)

Triage and Risk Assessment
Create a findings table focused on the OWASP Top 10.

Answers

1. **OWASP Risk Assessment Table**

OWASP Category	Vulnerability Example	Affected Application	Evidence (Your Lab)	Inherent Risk	Compliance Violation	Business Impact
A01: Broken Access Control	Direct access to administrative endpoints without authentication	BodgeIt Store	Attempted access to /bodgeit/admin.jsp without enforced authorization controls	High	PCI DSS 7.2.1 (Access Control), ISO 27001 A.9	Unauthorized access to admin functions, privilege escalation, exposure of sensitive data
A02: Cryptographic Failures	Transmission of sensitive data (credentials) over HTTP (plaintext)	WebGoat	Credentials observed in Burp HTTP history without encryption (no HTTPS enforcement)	High	PCI DSS 4.1 (Encryption in transit), GDPR Art. 32	Credential theft, session hijacking, account compromise

A03: Injection (SQL Injection)	Input fields vulnerable to SQL manipulation	BodgeIt Store (Search/Login)	Burp Intruder shows response length variations and abnormal responses for payloads	High	PCI DSS 6.5.1, OWASP ASVS V5	Full database compromise, data exfiltration, data manipulation
A05: Security Misconfiguration	Outdated and insecure services exposed	Entire BWA Server	Nmap shows outdated Apache, OpenSSL, expired SSL cert and TRACE method enabled	High	CIS Benchmarks, ISO 27001 A.12	Increased attack surface, exploitation of known CVEs, reputational damage
A07: Identification & Authentication Failures	Weak password policy and susceptibility to brute force	BodgeIt Store	Burp Intruder password attack showed uniform responses (no lockout, no rate limiting)	Medium –High	NIST CSF PR.AC-1, PCI DSS 8.3	Unauthorized account access, credential stuffing risk
A09: Security Logging & Monitoring Failures	Lack of detection for repeated malicious requests	Entire Application	No blocking or alerting observed during repeated Burp Intruder attacks	Medium	NIST CSF DE.CM-1, ISO 27001 A.12.4	Delayed incident detection, prolonged attacker dwell time

2. Executive Summary

To: Chief Information Security Officer (CISO) & Application Development Management
From: GRC Audit Team
Date: April 18th, 2026
Subject: OWASP Top 10 Risk Assessment – Legacy Web Applications (OWASP BWA Environment)

1. Overview

This assessment performed a targeted security evaluation of the **OWASP Broken Web Applications (BWA)** environment to determine its alignment with the organization's Application Security standards. Utilizing **Burp Suite Professional**, the evaluation moved beyond simple network reconnaissance into advanced application-layer testing, specifically focusing on the **BodgeIt Store** and **WebGoat** platforms.

The objective was to identify vulnerabilities within the **OWASP Top 10 (2021)** framework and translate these technical exploits into measurable business risks and compliance gaps.

2. Critical Findings & Technical Analysis

The assessment revealed a critical lack of secure coding practices and failed security control implementations across four primary categories:

A. **Broken Access Control (A01:2021):** Testing confirmed that administrative endpoints (e.g., /admin.jsp) were accessible to standard users without proper authorization. This represents a total failure of the **Principle of Least Privilege**, allowing for unauthorized data exfiltration and administrative privilege escalation.
B. **Cryptographic Failures (A02:2021):** Sensitive data, including user credentials, was captured in **plaintext** via Burp HTTP history. The absence of TLS/HTTPS encryption in transit violates fundamental data protection mandates (GDPR/PCI DSS) and exposes the organization to intercept-based attacks (Man-in-the-Middle).
C. **Injection (A03:2021):** Successful SQL Injection (SQLi) was confirmed via the search functionality using specialized payloads (' OR '1'='1 --). The application's failure to sanitize inputs allows an attacker to bypass authentication, manipulate the backend database, and access the entire customer inventory/user table.
D. **Identification & Authentication Failures (A07:2021):** Using **Burp Intruder**, a brute-force attack successfully identified weak credentials. The lack of account lockout policies and rate limiting indicates a significant

weakness in identity management, making the system susceptible to automated credential-stuffing attacks.

3. GRC & Compliance Implications

The identified vulnerabilities constitute direct violations of several regulatory and governance frameworks:

A. **PCI DSS 4.0 (Req. 4 & 6):** Failure to encrypt data in transit and presence of injection flaws.
B. **NIST CSF (PR.AC-4 / PR.DS-2):** Failure to manage access permissions and protect data integrity.
C. **ISO/IEC 27001 (A.14):** Inadequate security in development and support processes.

From a **Business Impact** perspective, the inherent risk is classified as **High**. A successful exploit of these vulnerabilities could lead to a total loss of **Confidentiality, Integrity, and Availability (CIA)**, resulting in regulatory fines, loss of customer trust, and significant remediation costs.

4. Strategic Recommendations

To mitigate these risks and move toward a compliant security posture, the following actions are recommended:

A. **Immediate Remediation (0-30 Days):** Enforce **HTTPS/TLS** across all web properties; implement **Parameterized Queries** (Prepared Statements) to stop SQL Injection; and restrict administrative URL access via server-side authorization checks.
B. **Operational Hardening (30-90 Days):** Implement **Account Lockout** and **Multi-Factor Authentication (MFA)**; upgrade outdated server components identified during Nmap reconnaissance; and establish a centralized logging and monitoring strategy (NIST DE.CM-1).
C. **Governance Integration (Long-Term):** Transition toward a **Secure Software Development Lifecycle (SSDLC)** by integrating DAST/SAST tools into the CI/CD pipeline and conducting mandatory OWASP-focused security training for development teams.

5. Conclusion

The BWA environment currently presents an unacceptable level of risk. The ease with which administrative access was bypassed and data was intercepted underscores a systemic failure in the current control environment. Immediate executive action is required to implement the proposed remediation plan and align the organization's digital assets with modern cybersecurity governance and compliance objectives.

References

1. **ICDFA.** (2026). *Governance, Compliance, and Risk (GCR) Program Curriculum: Web Application Testing for GRC Professionals*. International Cybersecurity and Digital Forensics Academy.
2. **Lyon, G.** (2026). *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*.
3. **National Institute of Standards and Technology (NIST).** (2020). *Security and Privacy Controls for Information Systems and Organizations* (NIST Special Publication 800-53, Revision 5). U.S. Department of Commerce.
4. **NIST.** (2024). *The NIST Cybersecurity Framework (CSF) 2.0*. Retrieved from https://www.nist.gov/cyberframework
5. **OWASP Foundation.** (2021). *OWASP Top 10:2021 - The 10 Most Critical Web Application Security Risks*. Retrieved from https://owasp.org/www-project-top-ten/
6. **OWASP Foundation.** (2026). *Broken Web Applications (BWA) Project Documentation*. GitHub Repository.
7. **PCI Security Standards Council.** (2022). *Payment Card Industry Data Security Standard (PCI DSS) v4.0*.
8. **PortSwigger.** (2026). *Burp Suite Professional Documentation: Using Intruder, Repeater, and Scanner*. Retrieved from https://portswigger.net/burp/documentation/desktop